

Problem 1

Problem 1

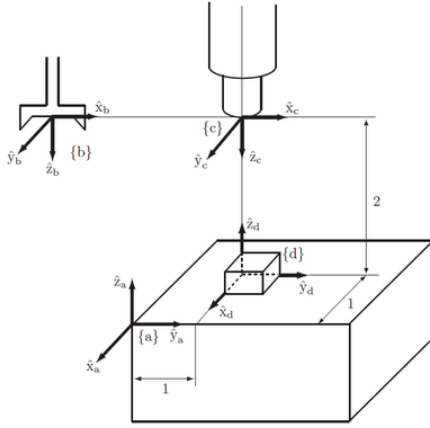


Figure 3.23: Four reference frames defined in a robot's workspace.

Four reference frames are shown in the robot workspace of Figure 3.23: the fixed frame {a}, the end-effector frame {b}, the camera frame {c}, and the workpiece frame {d}.

(a) Find T_{ad} and T_{cd} in terms of the dimensions given in the figure.

(b) Find T_{ab} given that

$$T_{bc} = \begin{bmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Part (a)

Frame {d} has the same orientation as {a}, so $R_{ad} = I$, and its offset can be read directly from the figure. Frame {d} is also unrotated relative to {c} up to the sign flip on $\hat{z}$, and its offset below {c} is the height 2:

$$T_{ad} = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_{cd} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Part (b)

Frame {d} is common to both the {a}-chain and the {b}-chain, so

$$T_{ab} = T_{ad} T_{dc} T_{cb}$$

where $T_{cb} = T_{bc}^{-1}$ and $T_{dc} = T_{cd}^{-1}$, using the rigid-body inverse $T^{-1} = \begin{bmatrix} R^T & -R^T P \\ 0^T & 1 \end{bmatrix}$:

$$T_{cb} = \begin{bmatrix} 1 & 0 & 0 & -4 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_{dc} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiplying $T_{dc}T_{cb}$ first,

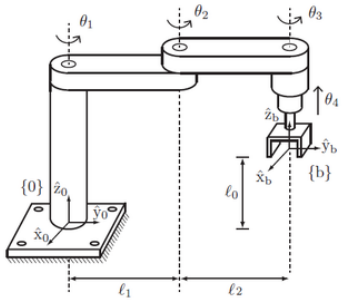
$$T_{dc}T_{cb} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & -4 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and then left-multiplying by T_{ad} gives

$$T_{ab} = \begin{bmatrix} 0 & 1 & 0 & -1 \\ 1 & 0 & 0 & -3 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problem 2

Problem 2



The RRRP SCARA robot in the figure is in its zero position. Determine the end-effector zero position configuration M , the screw axes S_i in $\{0\}$, and the screw axes B_i in b . For $\ell_0 = \ell_1 = \ell_2 = 1$ and the joint variable values $\theta = (0, \pi/2, -\pi/2, 1)$, use both the body form and the spatial form of the product of exponentials formula to find the transformation T .

With $\ell_0 = \ell_1 = \ell_2 = 1$, the zero-position end-effector configuration is

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \ell_1 + \ell_2 \\ 0 & 0 & 1 & \ell_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Screw Axes in $\{0\}$

The first three joints are revolute about $\hat{z}_0$, each located along the $\hat{x}_0$ axis at increasing offset; the fourth is prismatic along $\hat{z}_0$:

$$S_1 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ \ell_1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \quad S_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ \ell_1 + \ell_2 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 2 \\ 0 \\ 0 \end{pmatrix}, \quad S_4 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

with matrix forms

$$[S_1] = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad [S_2] = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$[S_3] = \begin{bmatrix} 0 & -1 & 0 & 2 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad [S_4] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Spatial Form of the Product of Exponentials

$$T(\theta) = e^{[S_1]\theta_1} e^{[S_2]\theta_2} e^{[S_3]\theta_3} e^{[S_4]\theta_4} M$$

with $e^{[S]\theta} = \begin{bmatrix} e^{[\omega]\theta} & (I\theta + (1 - \cos \theta)[\omega] + (\theta - \sin \theta)[\omega]^2)v \\ 0 & 1 \end{bmatrix}$, $e^{[\omega]\theta} = I + \sin \theta[\omega] + (1 - \cos \theta)[\omega]^2$,
and $\theta = (0, \pi/2, -\pi/2, 1)$.

Since $\theta_1 = 0$, $e^{[S_1]\theta_1} = I$. For $\theta_2 = \pi/2$,

$$e^{[S_2]\pi/2} = \begin{bmatrix} 0 & -1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For $\theta_3 = -\pi/2$,

$$e^{[S_3](-\pi/2)} = \begin{bmatrix} 0 & 1 & 0 & -2 \\ -1 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 4 is prismatic, so $e^{[S_4]\theta_4}$ is a pure translation by $v_4\theta_4$; with $\theta_4 = 1$,

$$e^{[S_4](1)} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Multiplying the four exponentials with M in order gives

$$T = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Body Form Check

The body screw axes are $\mathcal{B}_i = \text{Ad}_{M^{-1}} \mathcal{S}_i$, where

$$M^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \text{Ad}_{M^{-1}} = \begin{bmatrix} R & 0 \\ [p]R & R \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

since $R = I$ for M^{-1} . This gives

$$\mathcal{B}_1 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ -2 \\ 0 \\ 0 \end{pmatrix}, \quad \mathcal{B}_2 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ -1 \\ 0 \\ 0 \end{pmatrix}, \quad \mathcal{B}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad \mathcal{B}_4 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

Evaluating $T(\theta) = M e^{[\mathcal{B}_1]\theta_1} e^{[\mathcal{B}_2]\theta_2} e^{[\mathcal{B}_3]\theta_3} e^{[\mathcal{B}_4]\theta_4}$ with the same θ is guaranteed to reproduce the identical result, since $\mathcal{B}_i = \text{Ad}_{M^{-1}}\mathcal{S}_i$ by construction:

$$T = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problem 3

Problem 3

Write a program that solves the analytical inverse kinematics for a planar 3R robot with link lengths $L_1 = 3$, $L_2 = 2$, and $L_3 = 1$, given the desired position (x, y) and orientation θ of a frame fixed to the tip of the robot. Each joint has no limits. Your program should find all the solutions, give the joint angles for each, and draw the robot in these configurations. Test the code for the case of $(x, y, \theta) = (4, 0.5, 22^\circ)$ and the case $(x, y, \theta) = (1.5, -1.5, -105^\circ)$.

Inverse Kinematics Derivation

Subtracting off the last link along the desired tip orientation gives the wrist-center position, decoupling the problem into a planar 2R reach followed by an orientation adjustment:

$$\vec{\rho}_w = \vec{P}_E - L_3(\cos \theta \hat{x} + \sin \theta \hat{y})$$

The L_1 - L_2 - ρ_w triangle then gives β by the law of cosines, and α from the angle between L_1 and ρ_w :

$$L_1^2 + L_2^2 - 2L_1L_2 \cos \beta = |\rho_w|^2 \implies \beta = \cos^{-1} \left(\frac{L_1^2 + L_2^2 - |\rho_w|^2}{2L_1L_2} \right)$$

$$\alpha = \cos^{-1} \left(\frac{|\rho_w|^2 + L_1^2 - L_2^2}{2L_1|\rho_w|} \right), \quad \gamma = \text{atan2}(\rho_w)$$

and the joint angles follow directly:

$$\theta_1 = \gamma - \alpha, \quad \theta_2 = \pi - \beta, \quad \theta_3 = \theta - \theta_1 - \theta_2$$

Taking $\beta \rightarrow -\beta$ (equivalently $\alpha \rightarrow -\alpha$) flips the elbow and produces the second branch, so every reachable pose has exactly two analytical solutions. Both branches are implemented in **Problem 3.py** (Appendix) and checked against the forward-kinematics map for each test case.

Results

State	Solution	θ_1 [rad]	θ_2 [rad]	θ_3 [rad]
A: (4, 0.5, 22°)	1	-0.6297	1.8704	-0.8568
	2	0.7113	4.4128	-4.7400
B: (1.5, -1.5, -105°)	1	-1.0027	2.5011	-3.3309
	2	0.4131	3.7821	-6.0278

Table 1: Both analytical IK solutions for each test case; every row was confirmed to reproduce (x, y, θ) through the forward-kinematics map.

Problem 3: Planar 3R IK Solutions -- State A

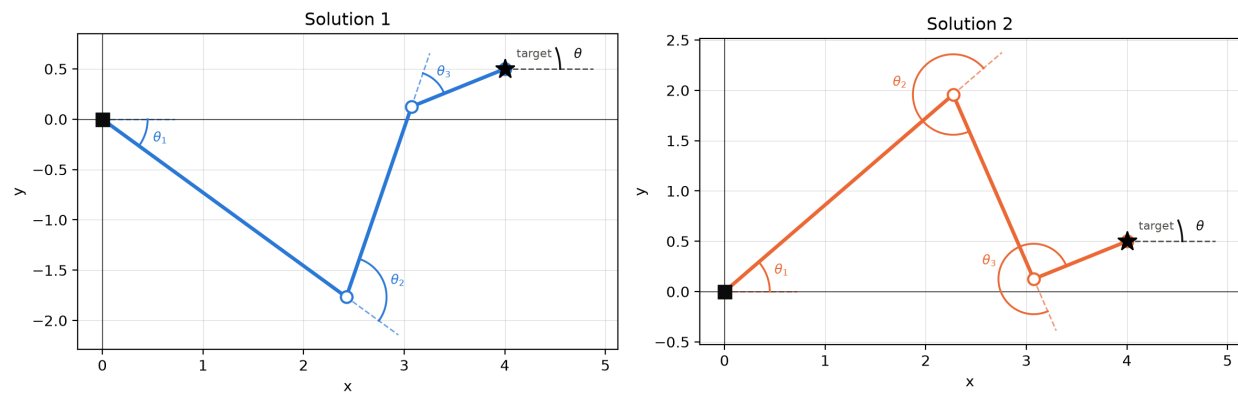


Figure 1: State A: $(x, y, \theta) = (4, 0.5, 22^\circ)$ – elbow-up and elbow-down solutions, with joint angles marked at each joint.

Problem 3: Planar 3R IK Solutions -- State B

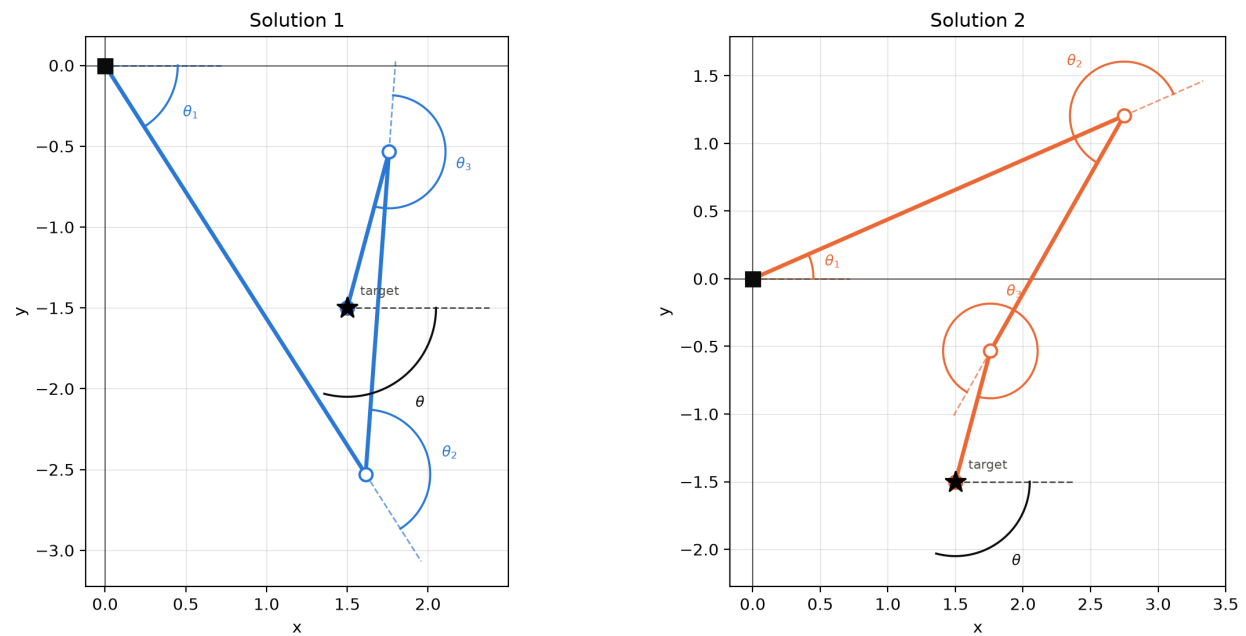


Figure 2: State B: $(x, y, \theta) = (1.5, -1.5, -105^\circ)$ – elbow-up and elbow-down solutions, with joint angles marked at each joint.

Appendix: Code Listings

Problem 3 Code

```

1
2 import os
3 import numpy as np
4 import matplotlib.pyplot as plt
5
6 FIG_DIR = os.path.join(os.path.dirname(__file__), "..", "latex", "Figures", "Graphs")
7 os.makedirs(FIG_DIR, exist_ok=True)
8
9
10 def savefig(fig, name):
11     fig.savefig(os.path.join(FIG_DIR, name), dpi=200, bbox_inches="tight")
12
13
14 L1 = 3
15 L2 = 2
16 L3 = 1
17
18 def getBeta(x , y):
19     num = L1**2 + L2**2 - x**2 - y**2
20     den = 2 * L1 * L2
21
22     beta = np.arccos(num/den)
23
24
25     return np.array((beta , -beta))
26
27
28 def getAlpha(x , y):
29
30     num = x**2 + y**2 + L1**2 - L2**2
31     den = 2 * L1 * np.sqrt(x**2 + y**2)
32
33     alpha = np.arccos(num/den)
34
35     return np.array((alpha, -alpha))
36
37
38 def forward_kinematics(theta1, theta2, theta3):
39
40     def transform(theta, L):
41         return np.array([
42             [np.cos(theta) , -np.sin(theta) , L*np.cos(theta)],
43             [np.sin(theta) , np.cos(theta) , L*np.sin(theta)],
44             [0 , 0 , 1]
45         ])
46
47     T1 = transform(theta1 , L1)
48     T2 = transform(theta2 , L2)
49     T3 = transform(theta3 , L3)
50
51     T = T1 @ T2 @ T3
52
53     x = T[0,2]
54     y = T[1,2]

```



```

55     theta = theta1 + theta2 + theta3
56
57     return np.array([x,y,theta])
58
59
60 def get_joint_positions(theta1, theta2, theta3):
61     p0 = np.array([0.0, 0.0])
62     p1 = p0 + L1 * np.array([np.cos(theta1), np.sin(theta1)])
63     p2 = p1 + L2 * np.array([np.cos(theta1 + theta2), np.sin(theta1 + theta2)])
64     p3 = p2 + L3 * np.array([np.cos(theta1 + theta2 + theta3), np.sin(theta1 + theta2 + theta3)])
65     return np.array([p0, p1, p2, p3])
66
67
68 def add_angle_arc(ax, center, start_angle, end_angle, radius, label, color="#52514e"):
69     # arc sweeping from the local horizontal (start_angle) to the link direction (end_angle)
70     t = np.linspace(start_angle, end_angle, 60)
71     ax.plot(center[0] + radius * np.cos(t), center[1] + radius * np.sin(t),
72            color=color, lw=1.3, zorder=6)
73
74     # dashed reference line = the "horizontal" this angle is measured from
75     ref_len = radius * 1.6
76     ax.plot([center[0], center[0] + ref_len * np.cos(start_angle)],
77            [center[1], center[1] + ref_len * np.sin(start_angle)],
78            ls="--", color=color, lw=1.0, alpha=0.7, zorder=2)
79
80     mid = (start_angle + end_angle) / 2
81     label_x = center[0] + radius * 1.35 * np.cos(mid)
82     label_y = center[1] + radius * 1.35 * np.sin(mid)
83     ax.plot(label_x, label_y, alpha=0) # register label position for autoscaling
84     ax.annotate(label, (label_x, label_y), ha="center", va="center",
85            fontsize=9, color=color, zorder=6)
86
87
88 def plot_solution_with_angles(ax, theta1, theta2, theta3, x, y, theta, color):
89     p0, p1, p2, p3 = get_joint_positions(theta1, theta2, theta3)
90
91     ax.plot([p0[0], p1[0], p2[0], p3[0]], [p0[1], p1[1], p2[1], p3[1]],
92            "-o", color=color, lw=2.5, ms=8, mfc="white", mec=color, mew=1.5, zorder=3)
93     ax.plot(p0[0], p0[1], "s", color="#0b0b0b", ms=9, zorder=4)
94     ax.plot(x, y, "k*", ms=14, zorder=5)
95     ax.annotate("target", (x, y), textcoords="offset points", xytext=(8, 8),
96            fontsize=8, color="#52514e")
97
98
99     add_angle_arc(ax, p0, 0.0, theta1, 0.45, r"$\theta_1$", color)
100    add_angle_arc(ax, p1, theta1, theta1 + theta2, 0.40, r"$\theta_2$", color)
101    add_angle_arc(ax, p2, theta1 + theta2, theta1 + theta2 + theta3, 0.35, r"$\theta_3$", color)
102    add_angle_arc(ax, p3, 0.0, theta, 0.55, r"$\theta$", "#0b0b0b")
103
104
105 def plot_state(name, x, y, theta, theta1_sols, theta2_sols, theta3_sols):
106     colors = ["#2a78d6", "#eb6834"]
107     n_sol = len(theta1_sols)
108
109     fig, axes = plt.subplots(1, n_sol, figsize=(6 * n_sol, 6))
110     if n_sol == 1:
111         axes = [axes]
112
113     for sol, ax in enumerate(axes):

```

```

114     plot_solution_with_angles(ax, theta1_sols[sol], theta2_sols[sol], theta3_sols[sol],
115                             x, y, theta, colors[sol % len(colors)])
116     ax.axhline(0, lw=0.5, c="k")
117     ax.axvline(0, lw=0.5, c="k")
118     ax.set_aspect("equal")
119     ax.set_xlabel("x")
120     ax.set_ylabel("y")
121     ax.set_title(f"Solution {sol + 1}")
122     ax.grid(True, lw=0.4, alpha=0.5)
123
124     fig.suptitle(f"Problem 3: Planar 3R IK Solutions -- State {name}")
125     fig.tight_layout()
126     savefig(fig, f"Problem3_{name}.png")
127     plt.close(fig)
128
129
130 def main():
131
132     states = {'A' : [4, 0.5 , 22] , 'B' : [1.5 , -1.5, -105]}
133
134     for name, state in states.items():
135         print(f"State {name}: {state}")
136         x = state[0]
137         y = state[1]
138         theta = np.radians(state[2])
139
140         x_w = x - L3 * np.cos(theta)
141         y_w = y - L3 * np.sin(theta)
142
143         gamma = np.arctan2(y_w, x_w)
144
145         alpha = getAlpha(x_w, y_w)
146         beta = getBeta(x_w, y_w)
147
148         theta1 = gamma - alpha
149         theta2 = np.pi - np.array(beta)
150         theta3 = theta - theta1 - theta2
151
152         for sol in range(len(theta1)):
153
154             print(f"Solution {sol+1}:")
155             print(f"Theta 1: {theta1[sol]}")
156             print(f"Theta 2: {theta2[sol]}")
157             print(f"Theta 3: {theta3[sol]}")
158
159             check = forward_kinematics(theta1[sol], theta2[sol], theta3[sol])
160             print(f"Solution Validation: {check}")
161
162         plot_state(name, x, y, theta, theta1, theta2, theta3)
163
164
165
166
167
168
169 if __name__ == "__main__":
170     main()

```

Listing 1: Planar 3R Inverse Kinematics